

Captura de eventos en IPTables

Detección de tráfico y notificación por correo electrónico

Jose Matías Ruiz Valero

1. Introducción

Este trabajo continúa el realizado anteriormente, en el que se montó un cortafuegos perimetral con IPTables sobre una red Cliente-Router-Servidor. En aquel trabajo se construyó listado básico y mínimo de reglas: políticas por defecto en DROP, reglas de filtrado por IP y puerto, NAT forwarding y una primera línea de logs. Ahora vamos a desarrollarlo aún más: vamos a crear un sistema observable, capaz de avisar cuando algo relevante sucede en la red.

El objetivo es ampliar la maqueta para identificar y señalizar eventos concretos del tráfico (por ejemplo, un intento de SSH no autorizado o una petición HTTP que cruza el NAT) y enviar una notificación a una plataforma externa. En lugar de disparar la ejecución de otro servicio, se enviará un correo a un administrador, demostrando que el evento ha sido capturado correctamente, pudiendo usarse para encadenar diferentes acciones.

No obstante, hay una restricción importante dicha en el documento de guía: IPTables vive en el espacio del kernel (ring 0), por lo que no es viable engancharle hooks directamente. Para identificar los eventos desde el espacio de usuario se han propuesto dos vías, que se desarrollan en este documento: analizar los mensajes de log que emite IPTables con prefijos personalizados, y/o consultar periódicamente sus contadores. Estaremos utilizando unos scripts que están publicados en el siguiente repositorio, de carácter interesante al lector: <https://github.com/JoseMatiasRuiz33/iptables-events>

2. Arquitectura del sistema

Se reutiliza la maqueta del trabajo anterior con las redes algo cambiadas, pues anteriormente borré las máquinas virtuales y cambié un poco los ajustes de adaptadores y demás: tres máquinas virtuales con Debian (64 bits, AMD, 2 GB RAM, 2 cores) bajo VirtualBox. El Cliente cuelga de la red WAN 192.168.2.0/24 con dirección 192.168.2.2, el Servidor de la red LAN 192.168.0.0/24, y entre ambos se sitúa el Router con dos interfaces (192.168.2.1 en la WAN y 192.168.0.1 en la LAN), donde corren tanto las IPTables como el nuevo monitor para la captura de los eventos.

La novedad respecto al trabajo anterior es el módulo de captura, que vive en el Router: un proceso en espacio de usuario que observa lo que produce IPTables y, cuando reconoce un evento conocido, llama al hook configurado para ese evento. En este trabajo, el hook abre una conexión SMTP con STARTTLS al servidor de correo externo de Gmail (pudiendo ser Outlook también) y entrega un mensaje al buzón del administrador. Lo vemos en el siguiente diagrama, las interfaces pueden variar, pero eso es lo de menos:

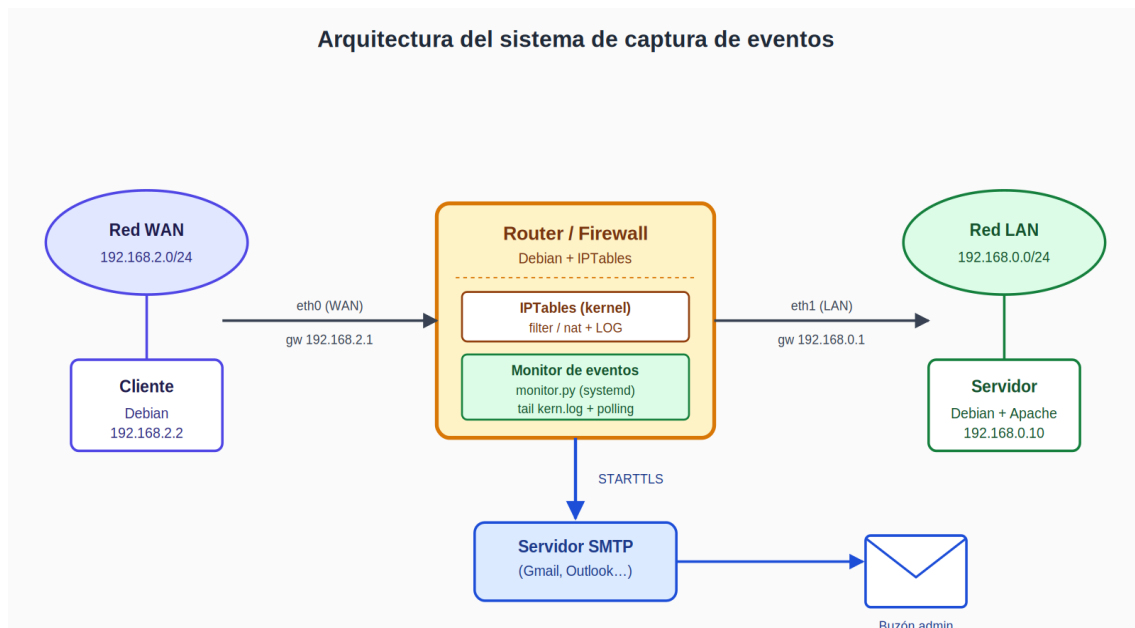


Figura 1. Arquitectura general: el monitor convive con IPTables en el Router y envía las notificaciones por SMTP al buzón del administrador.

3. Eventos a monitorizar

Se han escogido tres eventos representativos de las operaciones habituales del trabajo anterior. Cada uno se asocia a una regla concreta y a un prefijo de log único:

- **SSH bloqueado:** intento de conexión TCP al puerto 22 del Router desde una IP que no pertenece a la red de confianza. Prefijo: [EVT_SSH_DROP]
- **Acceso HTTP al servidor interno:** petición TCP al puerto 80 del Router que cruza el NAT hacia el Servidor. Prefijo: [EVT_HTTP_NAT]
- **Paquete rechazado por defecto:** cualquier paquete que cae al final de la cadena INPUT y es descartado por la política DROP. Prefijo: [EVT_DROP_DEFAULT]

El criterio para elegir un buen prefijo es que sea único, corto y fácil de filtrar con grep. Hay un espacio al final de los prefijos, este espacio final es importante: lo añadimos para que no se pegue al resto de los campos que IPTables vuelca a continuación (IN=, OUT=, SRC=, DST=, etc).

4. Configuración del envío de correo

El monitor no monta un servidor de correo propio: usa un servidor SMTP existente (Gmail, Outlook o cualquier otro), en nuestro caso Gmail. Esto simplifica la configuración y deja el problema de la entrega y la reputación en manos de un proveedor. Se ha elegido Gmail porque, a fecha de este trabajo, es el proveedor que sigue admitiendo autenticación SMTP por usuario y contraseña de aplicación de forma sencilla; otros proveedores como Outlook han retirado este método y exigen OAuth 2.0, lo cual

complicaría innecesariamente el script. Antes de tocar el Router se preparan tres cosas: la cuenta emisora, una contraseña de aplicación y el destinatario.

4.1 Cuenta emisora y contraseña de aplicación

Gmail no permite iniciar sesión SMTP con la contraseña habitual de la cuenta: hay que generar una “contraseña de aplicación”, una cadena de 16 caracteres específica para este uso y revocable de forma independiente. El procedimiento es el siguiente:

Activaremos la verificación en dos pasos en la cuenta emisora, crearemos la contraseña de aplicación mediante el siguiente link <https://myaccount.google.com/apppasswords> y tomaremos la contraseña que nos den de ejemplo (tipo abcd efgh ijkl mnop) sin espacios.

Los parámetros del servidor son fijos para Gmail: smtp.gmail.com en el puerto 587 con STARTTLS. La cuenta emisora (SMTP_USER y EMAIL_FROM) será mi cuenta de gmail; el destinatario (EMAIL_TO) es el buzón donde quieres recibir las alertas.

4.2 Fichero de configuración

Para hacer todo más sencillo, rápido y transparente, no escribiremos las credenciales directamente en el código, las guardaremos en /etc/iptables-monitor/config.env y las guardaremos como variables de entorno. El archivo tendrá la siguiente pinta:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=cuenta-ejemplo@gmail.com
SMTP_PASS=..... # contraseña de aplicación
EMAIL_FROM=cuenta-ejemplo@ejemplo.com
EMAIL_TO=cuenta-2@ejemplo.com
```

Un paso que podemos aplicar para tener una mayor seguridad es protegerlo con chmod, así le damos solo permisos al usuario root de leerlo. Además, en nuestro caso, el correo que envía y el correo que recibe será el mismo, para simplificar aún más el proceso de la Demo.

4.3 Prueba

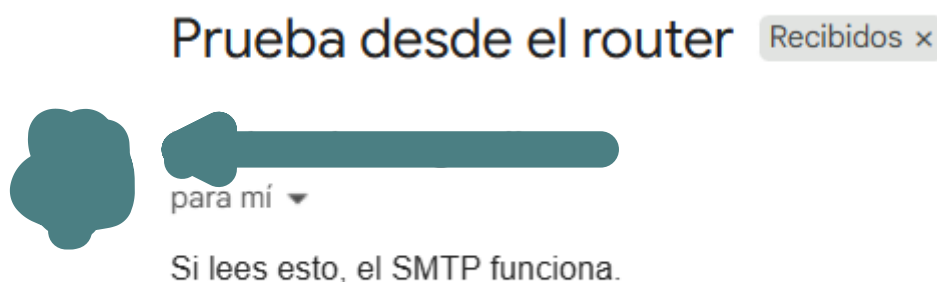
Antes de integrar nada en IPTables conviene confirmar que el Router puede entregar correo. Una prueba rápida desde Python:

```
python3 - <<'PY'
import os, smtplib, ssl, subprocess
from email.message import EmailMessage
env = dict(l.split('=',1) for l in open('/etc/iptables-monitor/config.env')
          if '=' in l and not l.startswith('#'))
msg = EmailMessage()
msg['From'] = env['EMAIL_FROM'].strip()
msg['To'] = env['EMAIL_TO'].strip()
msg['Subject'] = 'Prueba desde el router'
msg.set_content('Si lees esto, el SMTP funciona.')
with smtplib.SMTP(env['SMTP_HOST'].strip(), int(env['SMTP_PORT'])) as s:
    s.starttls(context=ssl.create_default_context())
    s.login(env['SMTP_USER'].strip(), env['SMTP_PASS'].strip())
```

```
s.send_message(msg)
PY
```

(cortesía de Deepseek, entiendo como funciona el código, simplemente lo hice así para agilizar el proceso, pues estos conceptos no van al caso del proyecto en cuestión, el código se encarga de leer el archivo que hemos puesto de script, formatearlo y pasar los parámetros para enviar un correo automatizado usando SMTP)

Y efectivamente, comprobamos que funciona:



5. Método 1: Detección por análisis de logs

La primera vía aprovecha la cadena LOG de IPTables. Como vimos en el trabajo anterior, una regla con `-j LOG` no detiene el paquete: solo emite una línea al log del kernel. Si esa línea lleva un prefijo único, un proceso en espacio de usuario puede leerlo con `tail -F` y reaccionar.

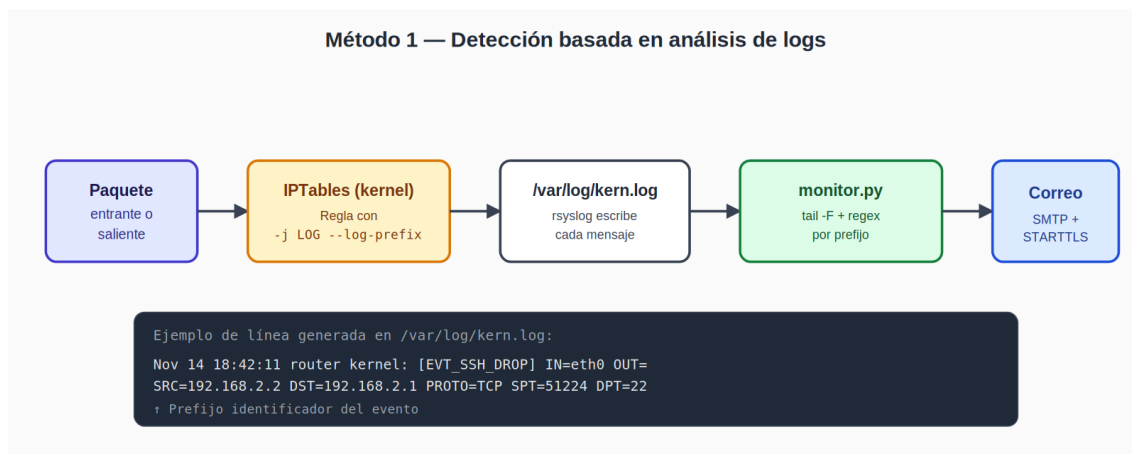


Figura 2. Flujo del método 1: la regla LOG marca cada paquete con un prefijo y el monitor lo identifica por regex.

5.1 Reglas con --log-prefix por evento

Sobre el cortafuegos del trabajo anterior, se insertan reglas LOG antes de la regla DROP correspondiente. La regla LOG debe ir siempre antes de la regla que descarta el paquete; si no, el paquete se descarta y nunca queda registro. Extracto del script:

```
# Ejemplo 1: SSH desde fuera de la red de confianza
iptables -A INPUT -p tcp --dport 22 -s 192.168.2.0/24 -j LOG --log-prefix
"[EVT_SSH_DROP] " --log-level info
iptables -A INPUT -p tcp --dport 22 -s 192.168.2.0/24 -j DROP
```

```
# Ejemplo 2: HTTP que cruza el NAT al servidor interno
iptables -A FORWARD -p tcp -d 192.168.0.2 --dport 80 -j LOG --log-prefix
"[EVT_HTTP_NAT] " --log-level info
iptables -A FORWARD -p tcp -d 192.168.0.2 --dport 80 -j ACCEPT
```

```
# Ejemplo 3: cualquier paquete que llega al final de INPUT de una red que no es de
confianza y se descarta
iptables -A INPUT -s 192.168.2.0/24 -j LOG --log-prefix "[EVT_DROP_DEFAULT] " --
log-level info

iptables -A INPUT -s 192.168.2.0/24 -j DROP
```

5.2 Script de monitorización log_monitor.py

El script abre /var/log/kern.log en modo seguimiento (tail -F) y, por cada nueva línea, comprueba si contiene alguno de los prefijos registrados. Si encuentra coincidencia, extrae los campos relevantes con una expresión regular y dispara el envío del correo.

Antes de lanzarlo, cambiamos las variables de entorno con los siguientes comandos:

```
set -a
source /etc/iptables-monitor/config.env
set +a
```

Cabe recalcar que en cada prueba ha habido ciertas modificaciones, además de eliminar las reglas IPTables anteriores o modificarlas por otras, las borraríamos con el comando iptables -F, o en el caso de contadores --zero. Esto es para probar cada una de ellas individualmente.

El script en cuestión es el siguiente:

```
#!/usr/bin/env python3
# "RUTA DE NUESTRO ARCHIVO"
import os, re, ssl, smtplib, subprocess
```

```
from email.message import EmailMessage
```

```
LOG_PATH = "/var/log/kern.log"
```

```
SMTP_HOST = os.environ["SMTP_HOST"]
```

```
SMTP_PORT = int(os.environ.get("SMTP_PORT", "587"))
```

```
SMTP_USER = os.environ["SMTP_USER"]
```

```
SMTP_PASS = os.environ["SMTP_PASS"]
```

```
EMAIL_FROM = os.environ["EMAIL_FROM"]
```

```
EMAIL_TO = os.environ["EMAIL_TO"]
```

```
EVENTS = {
```

```
    "EVT_SSH_DROP": "[ALERTA] Intento SSH bloqueado",
```

```
    "EVT_HTTP_NAT": "[INFO] Acceso HTTP por NAT",
```

```
    "EVT_DROP_DEFAULT": "[ALERTA] Paquete descartado por defecto",
```

```
}
```

```
FIELD_RE = re.compile(r"(SRC|DST|PROTO|SPT|DPT)=(\S+)")
```

```
def notify(subject, body):
```

```
    msg = EmailMessage()
```

```
    msg["From"], msg["To"], msg["Subject"] = EMAIL_FROM, EMAIL_TO, subject
```

```
    msg.set_content(body)
```

```
    with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=10) as s:
```

```
        s.starttls(context=ssl.create_default_context())
```

```
        s.login(SMTP_USER, SMTP_PASS)
```

```
        s.send_message(msg)
```

```
def handle(line):
```

```
    for key, subject in EVENTS.items():
```

```
        if f"[{key}]" in line:
```

```
            f = dict(FIELD_RE.findall(line))
```

```
            body = (f"Evento: {key}\n"
```

```
                    f"SRC: {f.get('SRC','')}\n"
```

```
                    f"DST: {f.get('DST','')}\n"
```

```
                    f"Proto: {f.get('PROTO','')}\n"
```

```
                    f"{f.get('SPT','')} -> {f.get('DPT','')}\n\n"
```

```
                    f"Línea original:\n{line}")
```

```
            notify(subject, body); return
```

```
with subprocess.Popen(["tail", "-F", "-n0", LOG_PATH],
```

```
                    stdout=subprocess.PIPE, text=True) as proc:
```

```
    for line in proc.stdout:
```

```
handle(line.strip())
```

Con las bases del anterior ejemplo no es difícil segmentar e interpretar el siguiente código, se encarga de filtrar los mensajes según qué evento suceda (EVENTS) y de mandarlo por correo, hacemos una prueba mandando un ping en la red prohibida **192.168.2.0/24** y vemos que funciona



El proceso se lanza con permisos suficientes para leer `/var/log/kern.log` (en Debian, que es el SO que estoy usando, basta con que sea ejecutado por root o por un usuario del grupo adm).

6. Método 2 — Detección por polling de contadores

La segunda vía no necesita leer ficheros: aprovecha que el propio módulo de IPTables lleva en el kernel un contador de paquetes y bytes por cada regla. Un proceso en espacio de usuario puede consultar esos contadores cada cierto tiempo y compararlos con la lectura anterior. Si han subido, es que la regla se ha disparado.

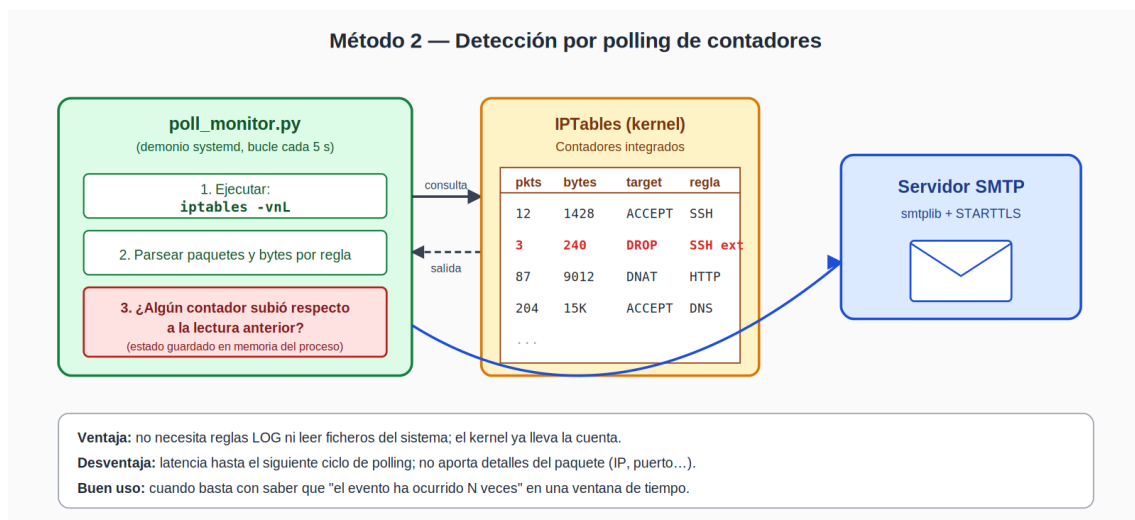


Figura 3. Flujo del método 2: el demonio consulta los contadores y compara con la lectura previa.

6.1 Etiquetar las reglas

Para que el monitor distinga unas reglas de otras sin acoplarse a su orden, conviene etiquetarlas con la extensión `-m comment`. Así podemos buscar por el comentario en la salida de `iptables -vnL`.

```
iptables -A INPUT -p tcp --dport 22 -s 192.168.2.0/24 -m comment --comment
"EVT_SSH_DROP" -j DROP
```

```
iptables -A FORWARD -p tcp -d 192.168.0.2 --dport 80 -m comment --comment
"EVT_HTTP_NAT" -j ACCEPT
```

```
iptables -A INPUT -s 192.168.2.0/24 -m comment --comment "EVT_DROP_DEFAULT" -j
DROP
```

6.2 Script de polling `poll_monitor.py`

El script ejecuta `iptables -vnL` cada cinco segundos, parsea la salida buscando las líneas que contienen el comentario de cada evento, y compara el contador de paquetes con el de la iteración anterior. Si ha crecido, dispara el correo con el delta. El script en cuestión:

```
#!/usr/bin/env python3
# "RUTA DE NUESTRO ARCHIVO"
import os, ssl, smtplib, subprocess, time
from email.message import EmailMessage

SMTP_HOST = os.environ["SMTP_HOST"]
SMTP_PORT = int(os.environ.get("SMTP_PORT", "587"))
SMTP_USER = os.environ["SMTP_USER"]
SMTP_PASS = os.environ["SMTP_PASS"]
EMAIL_FROM = os.environ["EMAIL_FROM"]
EMAIL_TO = os.environ["EMAIL_TO"]
INTERVAL = 5 # segundos

EVENTS = {
    "EVT_SSH_DROP": "[ALERTA] Intento SSH bloqueado (x{n})",
    "EVT_HTTP_NAT": "[INFO] Acceso HTTP por NAT (x{n})",
    "EVT_DROP_DEFAULT": "[ALERTA] Paquete descartado por defecto",
}

def read_counters():
    out = subprocess.run(["iptables", "-vnL"],
                        capture_output=True, text=True).stdout
    counters = {}
    for line in out.splitlines():
        for key in EVENTS:
            if f"/{key} *" in line:
                counters[key] = int(line.split()[0])
```



```

return counters

def notify(subject):
    msg = EmailMessage()
    msg["From"], msg["To"], msg["Subject"] = EMAIL_FROM, EMAIL_TO, subject
    msg.set_content(f'Detectado por polling de contadores.\n{subject}')
    with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=10) as s:
        s.starttls(context=ssl.create_default_context())
        s.login(SMTP_USER, SMTP_PASS)
        s.send_message(msg)

previous = read_counters()
while True:
    time.sleep(INTERVAL)
    current = read_counters()
    for key, template in EVENTS.items():
        delta = current.get(key, 0) - previous.get(key, 0)
        if delta > 0:
            notify(template.format(n=delta))
    previous = current

```

El método tiene menos detalle (no conoce la IP origen del paquete concreto), pero a cambio es mucho más sencillo y no depende del subsistema de logs. En la demostración se usan los dos métodos simultáneamente para que se vea cuál llega antes.

7. Comparación de ambos métodos

Ambos enfoques son válidos y complementarios. La elección depende de qué se quiera priorizar: el detalle del evento y la inmediatez frente a la sencillez y la independencia del subsistema de logs. Podemos contrastar ambos métodos de la siguiente forma.

Según su fuente de datos, el primer método (el de logs) lee las líneas de `/var/log/kern.log`, mientras que el segundo método lee los contadores nativos de `IPTables`, el primero ofrece una mayor cantidad de detalles con respecto al primero, además de tener una menor latencia (pues al escribir la línea se actualiza), este primero a su vez depende de las reglas `LOG` y de tener `rsyslog` activo, mientras que el método de polling únicamente requiere del binario de `IPTables`, el primer método no funcionaría si se cae el log mientras que el segundo podría perder eventos si ambos caen en el mismo ciclo de reloj (se solapan).

Los casos de uso son más particulares, el primer método podría usarse para dar alertas detalladas en tiempo real mientras que el segundo podría utilizarse para tener un conteo periódico del número de incidencias producidas. En un entorno real podrían combinarse; el método de logs para las alertas de seguridad que exigen detalle e inmediatez, y el de polling como red de seguridad para detectar reglas que se disparan aunque el sistema de logs falle.

8. Secuencia completa de un evento

El siguiente diagrama recoge la vida de un evento (intento de SSH desde el Cliente en la interfaz 192.168.2.2 hacia el Router en 192.168.2.1) desde que entra el paquete hasta que el administrador recibe el correo en su buzón. Se ilustra el método 1; en el método 2, el evento aparecería en la próxima lectura del polling.

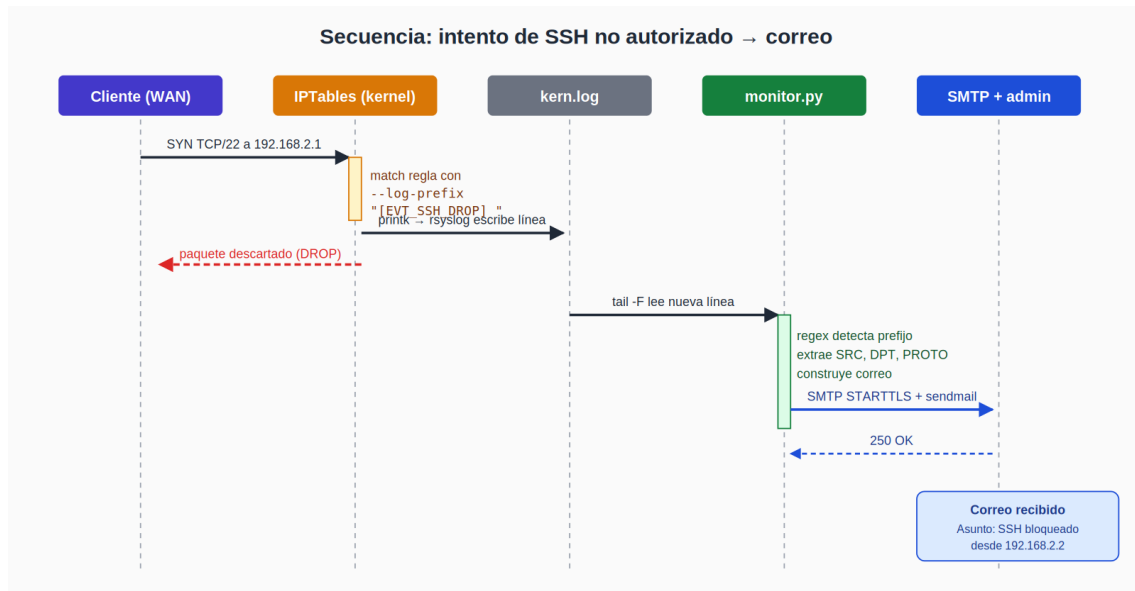


Figura 4. Secuencia detallada de un evento: el paquete dispara la regla LOG, el monitor lee la línea y envía el correo por SMTP.

9. Persistencia

Para que los monitores arranquen con el sistema y se reinicien si fallan, se empaquetan como unidades de systemd. Se muestra el caso del monitor de logs; el de polling es idéntico cambiando ExecStart.

```
# /etc/systemd/system/iptables-log-monitor.service
[Unit]
Description=IPTables event monitor (log-based)
After=network.target rsyslog.service
```

```
[Service]
Type=simple
EnvironmentFile=/etc/iptables-monitor/config.env
ExecStart=/usr/bin/python3 nuestra_ruta_del_script
Restart=on-failure
RestartSec=3
User=root
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Activación e inspección:

```
sudo systemctl daemon-reload  
sudo systemctl enable --now iptables-log-monitor.service  
sudo systemctl status iptables-log-monitor.service  
journalctl -u iptables-log-monitor.service -f
```

Con esto daría por finalizada la memoria del segundo trabajo de Gestión y Administración de Redes, ha resultado de carácter muy interesante y educativo el poder automatizar el envío de correos de esta forma, se me ocurren un montón de aplicaciones para sistemas informáticos relacionadas a la seguridad de estos, además de otras aplicaciones de carácter más relacionado al marketing, por ejemplo, automatizar el envío de correos relacionados a cierto producto y/o por una suscripción de un cliente que esté a punto de agotarse... Las posibilidades son infinitas.